

Distributed Configuration Updater: Design and Implementation of a CRDT-Based Settings Replication System

Gabriele Da Re
Dept. Mathematics
Università degli Studi di Padova
Padova, Italy
gabriele.dare00@gmail.com

Abstract—This paper describes the architecture, implementation and decisions made during the creation of a configuration synchronization system for small server clusters. The system aims to replicate Key-Value (KV) entries across nodes through: Last-Writer-Wins (LWW), Conflict-free Replicated Data Type (CRDT), gRPC transport protocol and a Hybrid Logical Clock (HLC) for causal ordering. We explain the decisions made designing each module and the problems encountered/mitigated during development such as: Clock-skew, crash persistence handling, tombstones management and the anti-entropy approach and lifecycle enforcement between initialisation and runtime phases.

Index Terms—CRDT, distributed systems, data consistency, hybrid logical clocks, gRPC, Go, key-value store

I. INTRODUCTION

Configuration management across a server cluster is a common operational challenge. Changes to settings such as feature flags, service endpoints, parameters must propagate to every node without a coordinator becoming the single failure point.

Approaches like etcd, Consul rely on a leader node or a distributed database. These provide linearisability at the cost of availability (CP part of the CAP theorem and ACID principle): when the leader or a majority quorum is unreachable, there is no working system. For configuration data, human operators can intervene on conflicts and having stale data in some circumstances is relatively low risk, thus the trade-off favours Availability and Partition tolerance having **eventual** Consistency (AP).

Conflict-free Replicated Data Types (CRDTs) [1] offer an alternative: every node can accept writes locally, the data structure guarantees that any two nodes with the same set of updates will converge to the same state, without coordination. This paper describes a practical implementation of a CRDT based configuration store targeting small, fixed-membership clusters where $O(100)$ configuration keys are replicated across $O(10)$ nodes. This limit can be pushed up to $O(1000)$ keys and $O(50)$ nodes if necessary, based upon limits imposed from the full snapshot, sync implementation VII.

The implementation is written in Go and uses gRPC for inter-node communication. The remainder of this paper is structured as follows. Section II presents the overall system

architecture. Section III describes the CRDT data model. Section IV covers the Hybrid Logical Clock. Section V explains the concurrency model. Section VI details the network layer. Section VII discusses anti-entropy reconciliation. Section VIII describes crash-safe persistence. Section IX presents tombstone garbage collection. Section X covers lifecycle and error handling. Section XI outlines the testing strategy. Section XII concludes.

II. SYSTEM ARCHITECTURE

The system is structured as five Go packages with a strict dependency hierarchy, shown in Table I.

Table I
PACKAGE RESPONSIBILITIES

Package	Responsibility
types	Shared domain types: HLC, SettingEntry, Snapshot
crdt	LWW CRDT, state machine, persistence
logic	Local settings file watcher and writer
network	gRPC server, streaming client, proto conversion
controller	Wires all packages, lifecycle and backgrounds tasks
main	CLI entry-point

The data flows through the system following four paths, wired up in `controller.Run`:

- 1) **Local write**: the `logic` watcher detects file changes on write and pushes a `SettingEntry` to `crdt.NotifyLocal`, which queues it in local channel of the CRDT actor. The CRDT ticks its clock, timestamps the entry and merges it, persists state, and calls `OnBroadcast` delegating the broadcast to the network layer.
- 2) **Remote write**: a peer to which the node is subscribed streams an update, `network.Client` calls `crdt.NotifyRemote`. The CRDT updates its clock, merges, persists, and calls `OnFileSync` that delegates the `logic` layer to write the change locally.
- 3) **Anti-entropy**: a ticker in the controller calls `syncAllPeers`, which sends the local snapshot

to each peer via a Sync RPC whose returns (if any) any newer entries through NotifyRemote.

- 4) **Tombstone GC**: another separate ticker calls `crdt.PurgeTombstones`, whose adhibited at removing stale deleted entries from the in-memory state map via the CRDT actor model.

The controller is the only package that depends on all the others, preventing import cycles.

III. CRDT DATA MODEL

A. Data Model

The data structure applies **LWW-Register** semantics [1] per key in a dictionary: each `SettingEntry` carries a payload, an HLC as timestamp and a tombstone flag. The tombstone turns each register into a two-state register (live/deleted), deletion itself is a timestamped write, subject to the same LWW rule.

```
type SettingEntry struct {
    Key      string
    Value    string
    Clock    HLC
    Deleted  bool
}
```

B. Merge Semantics

Two nodes merge by iterating one map against the other and checking for each key the entry with the higher clock thus the winning one. On equal clocks, the lexicographically larger NodeID is used as tiebreaker, ensuring deterministic total order.

```
func (c *CRDT) merge(incoming SettingEntry) bool {
    existing, ok := c.state[incoming.Key]
    if ok && !existing.Clock.Before(incoming.Clock) {
        return false
    }
    c.state[incoming.Key] = incoming
    return true
}
```

C. Tombstone Semantics

Deletions are represented as tombstones: entry with `Deleted = true` and a fresh clock. Tombstones undergo the same LWW merge as the payload and can be displaced by a later write. This prevents the anomaly where a deletion arrives after a concurrent write and erroneously removes the newer value.

D. CvRDT

The implementation is a hybrid CRDT: each local state change broadcasts a `SettingEntry` to peers (operation-based style), while a timed Sync provides anti-entropy reconciliation (state-based style). Because LWW merge is commutative and idempotent, each broadcast is algebraically equivalent to a partial state merge. Convergence is guaranteed entirely by the state-based anti-entropy, the broadcast is a best-effort optimisation.

E. Convergence Proof (CvRDT)

a) *State forms a join-semilattice.*: Define the replica state as $S = \text{Map}[\text{String} \rightarrow \text{SettingEntry}]$. Impose a partial order $\forall k \in \text{Keys}$

$$S_1 \leq S_2 \iff \forall k : S_1[k].\text{Clock} \leq S_2[k].\text{Clock} \quad (1)$$

where `Before` defines the strict order $<$ on clocks (Definition IV), and $\leq$ is its reflexive closure: $a \leq b \iff a < b \vee a = b$. The least upper bound per key is

$$(S_1 \sqcup S_2)[k] = \arg \max_{\text{Clock}} (S_1[k], S_2[k]) \quad (2)$$

which is exactly `merge()`.

b) *Merge implements the LUB.*: Let A, B, C be `SettingEntry` values for the same key.

- **Commutative**: $\text{merge}(A, B) = \text{merge}(B, A)$, since $\max(\text{Clock}_A, \text{Clock}_B)$ is symmetric.
- **Idempotent**: $\text{merge}(A, A) = A$, since $\max(T, T) = T$.
- **Associative**: $\text{merge}(A, \text{merge}(B, C)) = \text{merge}(\text{merge}(A, B), C)$, since $\max$ is associative.

c) *Monotonicity.*: Every merge moves state upward in the lattice:

$$S \leq \text{merge}(S, e), \quad (3)$$

since `merge` replaces an entry only when the incoming clock is strictly greater, and never decreases a clock value.

d) *Liveness.*: `syncAllPeers` runs periodically exchanging full snapshots. After a network partition, reconnection triggers Sync, delivering all missing updates. Hence every update eventually reaches every replica.

e) *Conclusion.*: By the Eventual Consistency theorem for CvRDTs [1]: any two replicas that have received the same set of updates hold identical maps, regardless of delivery order.

IV. HYBRID LOGICAL CLOCK

A. Motivation

A pure Lamport clock provides causal ordering but loses the physical-time locality property: events that happen close together in real time may receive widely separated clock values if logical increments accumulate. Hybrid Logical Clocks [2] integrates the Lamport clock with a wallTime, preserving the benefits of both.

B. Clock Structure

Each HLC is a triple $(\text{wall}, \text{logical}, \text{nodeID})$. The wall field stores Unix nanoseconds, the logical field is a 32-bit logical counter, the NodeID is a UUID string adhibited for the tiebreak. III-B. This ensures `Before()` commutativity since $\forall n_i, n_j \in \text{Nodes}, i \neq j : \text{Id}(n_i) \neq \text{Id}(n_j)$

C. Tick and Update Rules

On a local event (*tick*), the clock advances (ℓ_l, ℓ_r for $logical_{local}, logical_{received}$, w_l, w_r for $wall_{local}, wall_{received}$):

$$\begin{aligned} wall' &= \max(w_l, now), \\ logical' &= \begin{cases} logical + 1 & wall' = w_l \\ 0 & otherwise \end{cases} \end{aligned} \quad (4)$$

On receiving a remote clock (*update*), the canonical Kulkarni–Demirbas rule [2] is applied:

$$wall' = \max(w_l, w_r, now) \quad (5)$$

The logical counter then captures concurrency at the chosen wall time :

$$logical' = \begin{cases} \max(\ell_l, \ell_r) + 1 & w' = w_l = w_r \\ \ell_r + 1 & w' = w_r \neq w_l \\ 0 & otherwise \end{cases} \quad (6)$$

D. Clock Skew Handling

```
if received.WallTime - now > int64(1*time.Second) {
    return fmt.Errorf("rejecting: clock too far in
        future")
}
```

Rejection maps cleanly onto two failure cases.

a) *Sender clock too far ahead*: (e.g. B at +6s, **threshold** 5s, in our case 1s): every correctly synchronised node rejects the entry. B's writes are dropped cluster-wide until its NTP is corrected. No divergence occurs

b) *Receiver clock too far behind*: (e.g. A entering with -3s, B entering with +4s): from A's perspective the skew is 7s > 5s, so A rejects, from a synchronised node/cluster C the skew is 4s < 5s, so C accepts and updates the clock to B wall skewing the whole cluster HLC to B's +4s until next NTP correction and cluster making up for those seconds. A and C temporarily diverge. Once A's clock is corrected, the next anti-entropy round syncs A to the node/cluster C + B. The divergence window is bounded by the NTP correction time, not permanent.

The 1s threshold is an operational boundary that is within the 100ms NTP average skew. Nodes whose NTP drift pass it, diverge until clock correction, this is resolved at the infrastructure level (NTP monitoring, alerting) rather than silently accepting future-stamped entries that would risk clock poisoning across the cluster.

V. CONCURRENCY MODEL

All mutations to the CRDT state map are serialised through a single actor goroutine running a `select` loop over five typed channels:

```
type CRDT struct {
    state      map[string]SettingEntry
    localCh    chan SettingEntry
    remoteCh   chan SettingEntry
    queryCh    chan queryRequest
    snapshotCh chan snapshotRequest
    gcCh       chan int64
}
```

No mutex is required: only the actor goroutine accesses channels and CRDT state ensuring operation atomicity. External callers (`NotifyLocal`, `NotifyRemote`, `Snapshot`, `queryRequest`, `PurgeTombstones`) communicate exclusively through channels. `Snapshot` and `query` requests are buffered synchronous response channels thus blocking until the request is serviced.

Callbacks (`OnBroadcast`, `OnFileSync`) are invoked synchronously within the actor loop. The `OnFileSync` callback dispatches disk writes on a separate goroutine to prevent blocking the actor loop on I/O.

VI. NETWORK LAYER

A. gRPC Service

The inter-node protocol exposes two RPCs over gRPC:

- **SubscribeStateUpdates** (server-streaming): the caller receives `ServerStateUpdate` messages through a persistent stream every time the server calls `Broadcast`. This ensures real-time propagation of local edits.
- **Sync** (unary): the caller sends its local snapshot, the server updates its entries and responds with local server side entries that are newer. Used for anti-entropy reconciliation.

The server maintains a map of subscriber channels protected by a mutex. `Broadcast` under read-lock retrieves the subscribed peers channels, then sends the update to each channel in a non locking manner. Slow subscribers drop updates relying on the next anti-entropy pass to recover.

B. Abstraction Boundary

A design decision was taken such all protobuf conversion (`ToProto`, `FromProto`, `SnapshotToProto`) is confined in the `network` package. The `Client` methods accept and return domain types (`types.Snapshot`, `types.SettingEntry`). Conversion to and from wire format is performed internally.

This was not in the original design: an earlier iteration of `Client.Sync` and `(Client.Broadcast)` accepted `[]*userpb.SettingEntry` and `[]*userpb.ServerStateUpdate`, thus requiring the controller to perform the conversion before each call. Refactor moved conversion dependency into `network` package, making the transport layer a true black box.

C. Reconnection and Keepalives

`Subscribe` method in `Client` wraps the streaming RPC in a retry loop: on any error, including server restart or network interruption, it waits three seconds and tries to re-establish the stream, first of all performing a `Sync` call to replay any entries missed during the outage. gRPC keepalive parameters are configured to detect dead connections.

VII. ANTI-ENTROPY RECONCILIATION

Broadcast over gRPC is best-effort: a slow, offline or reconnecting subscriber may miss updates. A (state-based) CRDT that relies solely on broadcast for propagation is not eventually consistent in presence of message losses, nodes that miss a broadcast permanently diverge.

Production CRDT deployments (Riak, Dynamo [3]) address this with a background anti-entropy mechanism: periodic full-state comparison identifies and repairs divergence independently of the broadcast path (Riak AAE).

This system implements a lightweight variant: `syncAllPeers` is called periodically by the controller, this sends a local snapshot to each peer via the `Sync` RPC, updating peer entries and also local entries on returned values. This is sufficient for eventual consistency: even if every broadcast is dropped, the state will converge within one anti-entropy interval.

There is an unimplemented enhancement regarding exchanges of unupdated KV pairs: currently can happen that 2 nodes at the same instant send a `Sync` request, this will on each node take a snapshot of local state and send it, then upon receiving the snapshot both nodes update their local entries, both send the updates that consider necessary to the other that already had applied them through the received snapshot. This is solvable but out of context for this project purpose. (eg. Jitter, follower/leader `Sync`, Merkle tree, Delta state ecc.)

```
func (ctrl *Controller) syncAllPeers(ctx context.
Context) {
    snap := ctrl.crdt.Snapshot()
    for _, client := range ctrl.clients {
        go client.Sync(ctx, snap, func(e
            SettingEntry) {
                ctrl.crdt.NotifyRemote(e)
            })
    }
}
```

A 60-second interval was chosen as default: bounding divergence to one minute in worst-case broadcast loss and generating negligible network traffic for the cluster size targeted.

VIII. CRASH-SAFE PERSISTENCE

After each merge, CRDT serialises its state to `crdt_state.json` found in the CRDT working directory. The naive approach using `os.WriteFile` is not crash-safe: if the process is killed after `WriteFile` had truncated the file and before the kernel had flushed all pages to disk, the node restarts with a corrupted state file.

The correct pattern we used is the atomic rename:

```
func atomicWriteFile(path string, data []byte) error
{
    tmp, err := os.CreateTemp(filepath.Dir(path), ".
    tmp_*")
    // ... write data to tmp, call tmp.Sync(), tmp.
    Close() ...
    return os.Rename(tmp.Name(), path)
}
```

Three properties make this safe:

- 1) The temporary file is in the same directory of the target, thus the rename is within a single filesystem so, atomic at VFS layer on POSIX systems.
- 2) `tmp.Sync()` flushes file data to storage before the rename, preventing data to be in the page cache during power-loss.
- 3) `os.Rename` is atomic on POSIX: any reader will see either the old file or the new file, never a partially-written intermediate state.

If the process crashes before the rename, the temporary file is left in the directory and cleaned up during the next `crdt.Init()` call, the original file remains intact.

If the crash happens after state is persisted, `crdt.Reconcile` function is called one time in `controller.Run()` thus fully restoring local file integrity on startup. This is a limiting factor since on an already initialized node, if edits are made on the setting file, while the node is offline, these will be discarded in favour of last known state.

IX. GARBAGE COLLECTION

A. Problem

Deleted entries persist in CRDT state file indefinitely, this also happens for **temporary files** generated by `atomicRead` upon system crash. In systems with frequent delete-recreate cycles over large KV stores, tombstone accumulation brings two problems: memory leak over time, slower `Sync` and `crdt.Run` since larger snapshots are required.

B. Init cleanup

With regard to orphaned tmp files from `atomicRead`, the best solution in this case was also the easiest. Removal of all pattern matched tmp files on `crdt.Init` call.

C. TTL-Based Approach

The system also implements a TTL-based garbage collector. A ticker in the controller calls `crdt.PurgeTombstones(ttl)` passing the TTL. The CRDT actor then processes the request from `gcCh` channel monitored in its `Run` loop, thus maintaining the single-writer on `c.state`, removing tombstones with `wallTime` older than `now - ttl`.

```
func (c *CRDT) purgeTombstones(ttl int64) {
    diff := time.Now().UnixNano() - ttl
    purged := 0
    for key, entry := range c.state {
        if entry.Deleted && entry.Clock.WallTime <
            diff {
                delete(c.state, key)
                purged++
            }
    }
    if purged > 0 {
        c.saveState()
        log.Printf("tombstone GC: purged %d entries
            ", purged)
    }
}
```

The default TTL is 14 days, twice the intended maximum node downtime after which an operator would investigate. This ensures that tombstones had been observed and applied by all healthy nodes via anti-entropy before it is removed. After a week of downtime a node supposedly needs a re-initiation.

D. Safety Considerations

The TTL approach has one known failure point: if a node is offline for longer than the TTL and holds a non-deleted entry for a key tombstoned on all other nodes, the node will republish entry thus the deletion will appear to be reversed. This is solved by forcing the node to re-initialize with empty settings wether on startup the node follows `time.Now() - lastShutdownTime > TTL`

There are also 2 known limitations we call sputtering cycle, determined by clock skew on machines running a tombstone garbage collection, then performing a Sync request. This will, from the skewed node, propagate again the GC'd entry. Moreover if a Sync request is performed just before a GC message is processed in the CRDT actor, Sync cycle brings with it GC'd entries creating a the same sputtering problem. These are mitigated through wire-layer filtering in the Sync handler. Before including a local entry in the response or applying an incoming remote entry, the handler skips any entry where `Deleted = true` and `now - wallTime > TTL`. Filtering is applied in both directions preventing resurrection of already-collected tombstones regardless of clock skew or snapshot-before-GC ordering between nodes.

X. LIFECYCLE AND ERROR HANDLING

A. Init / New Separation

The CRDT package exposes three functions:

- `New(workDir)` instantiates a `*CRDT` value with buffered channels. Pure allocation, no I/O, no error.
- `Init()` loads an existing node: cleans up any orphaned atomic-write tmp files, reads `node_id`, and deserialises `crdt_state.json`. Returns an error if the working directory has not been initialised (“run init first”).
- `InitNew(entries)` bootstraps a fresh node: creates the working directory, writes `node_id` if absent, stamps all supplied entries with a fresh HLC tick, and persists the resulting state. Used both at first-time setup and when forcing a clean slate.

The controller decides which path to take via `fixNodeState`:

```
func (ctrl *Controller) fixNodeState(lastShutdown
int64) error {
    if lastShutdown != -1 &&
        time.Now().UnixNano()-lastShutdown > ctrl.
            tombstoneTTL {
        return ctrl.crdt.InitNew(types.Settings{})
    }
    return ctrl.crdt.Init()
}
```

`checkLastShutdown()` returns `-1` when no shutdown file exists (first startup after `InitNew` or `InitEmptyNode`), causing `Init()` to load the freshly written state. If the

node was offline longer than the tombstone TTL, `InitNew` reinitialises with an empty state making the node re-sync from peers rather than republishing entries already garbage collected cluster wide. `saveShutdownTime()` is deferred in `Run` and records the clean-shutdown timestamp used by the next invocation.

B. Error Propagation Principle

Functions in library packages (`crdt`, `network`) return errors rather than calling `log.Fatal` or `os.Exit` directly. `log.Fatal` belongs only at the application boundary (`main`) where there is sufficient context to decide that an error is truly unrecoverable. This makes every package independently testable without the test process exiting.

XI. TESTING STRATEGY

A. Clock Injection

The HLC struct accepts a `Clock` interface via parametrized options at construction time. Tests inject `MockClock` to control `wallTime` advancement deterministically, covering edge cases such as backward clock steps, simultaneous ticks, and the case where system clock overtakes both local and received `wallTime`, resetting logical counter to zero.

B. Network Tests with bufconn

The `network` package is tested with `google.golang.org/grpc/test/bufconn`, an in-memory listener that presents a real gRPC connection without opening any network socket. This allows tests to exercise the full gRPC stack without port allocation or network related problems.

C. Coverage

Line coverage targets were established and maintained: types at 100%, `crdt` at approximately 90%, `logic` at approximately 90%, `network` at approximately 91%, and `controller` at approximately 92%. The remaining uncovered branches are structurally untestable without process-level mocking, `json.Marshal` on `map[string]string` (the error path never fires in the Go standard library), and the `fsnotify` closed-channel path (fires only after the watcher goroutine has already exited).

XII. CONCLUSION

This paper presented a production-quality CRDT-based configuration synchronisation system. The core LWW CRDT provides convergence under concurrent writes and network partitions. The actor concurrency model eliminates lock contention. Anti-entropy reconciliation bounds divergence to Sync interval regardless of broadcast reliability. Atomic persistence through `os.Rename` prevents corruption on ungracefull shutdown. Tombstone GC fixes CRDT state growth for long-running deployments.

Several design decisions evolved during implementation. The protobuf abstraction boundary was moved into the `network` package, removing transport-layer knowledge from

the controller. Error propagation got modified to return `error` from library code and keep `log.Fatal` in the application entry-point. Forcefull init had ben added for nodes rejoining the cluster after more than TTL elapsed since last shutdown. A full reconciliation cycle had ben added upon start to make sure ungracefull shutdowns did not produce stale entries.

The system does not address gossip-based broadcast efficiency, or metrics observability, all known gaps appropriate for future work in production deployments at larger scale.

REFERENCES

- [1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “A comprehensive study of convergent and commutative replicated data types,” INRIA, Tech. Rep. RR-7506, 2011.
- [2] S. S. Kulkarni and M. Demirbas, “Logical physical clocks and consistent snapshots in globally distributed databases,” in *Proc. 18th Int. Conf. Principles of Distributed Systems (OPODIS)*, Cortina d’Ampezzo, Italy, Dec. 2014.
- [3] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proc. 21st ACM Symp. Operating Systems Principles (SOSP)*, Stevenson, WA, Oct. 2007, pp. 205–220.